



STORYBOOK GENUI

DOCUMENTO DE CONTINUIDADE + WORKFLOW

OpenCode · Android/Termux · UX=DX · Claude Chatlogs · GitHub

v1.0 · Joaonzin · BH/MG · 2026 · compilerum/storybook-genui

✓ Storybook 8.6.18	✓ Android localhost:6006	✓ Monorepo pnpm+Turbo	⌚ Primitives + GenUI	⌚ PWA + GraphQL
-----------------------	-----------------------------	--------------------------	-------------------------	--------------------

Prova de Conceito Confirmada: O screenshot mostra Storybook 10.3 rodando no browser Quetta via **localhost:6006**, viewport Mobile S 360x640, story Card/Mobile Default ativa — pipeline Termux → proot-distro Ubuntu → Node → browser Android funcionando.

0. FILOSOFEMA — UX = DX

« A experiência do desenvolvedor é a experiência do usuário em gestação. O componente difícil de compor é o componente difícil de usar. O workflow que irrita o dev produz a interface que irrita o usuário. UX = DX não é metáfora — é equação causal. »

Formalismo: Seja UX(u) a experiência do usuário u e DX(d) a experiência do dev d. Afirma-se: **UX(u) = f(DX(d))** — ou seja, a qualidade da interface é função direta da qualidade do processo de desenvolvimento. Portanto, otimizar DX é otimizar UX por indução.

UX Research integrado ao DX ▪ Storybook story = protótipo testável pelo usuário (não só pelo dev) ▪ Viewport 360px default = empatia com 80% dos usuários reais no Brasil ▪ Mobile-first constraint = decisão de UX: hierarquia de informação forçada ▪ Story docs autodocs = spec viva acessível por designers e PMs ▪ A11y addon = acessibilidade testada na fonte, não no final	DX como UX Research method ▪ Friction log: anotar toda vez que o dev freia — essa é dor de usuário latente ▪ Component API review: props confusas = interação confusa ▪ Build time como métrica: dev lento = iteração lenta = UX estagnada ▪ README como onboarding test: se dev novo trava, usuário novo trava ▪ Error messages: mensagem de erro ruim para dev = feedback ruim para user
---	---

1. ESTADO ATUAL — O QUE JÁ EXISTE

ELI5: Aqui está o inventário do que já está de pé antes de continuar.

1.1 Infra Confirmada (screenshots validam)

compilerum/storybook-genui ▪ Storybook 8.6.18 rodando em Android ▪ pnpm workspaces + Turborepo ▪ 7 pacotes: core, ui, gen-ui, server-ui, graphql, pwa, emacs-bridge ▪ Card component + 3 stories (Docs, Elevated, Mobile Default) ▪ Rollup WASM patch (android-arm64 workaround) ▪ Viewport default: Mobile S 360x640	Documentos gerados: ▪ PLANO_ENGENHARIA_PROMPTS.md — arquitetura completa ▪ README-GenUI.md — onboarding ▪ session-genUI.md — chatlog OpenCode completo ▪ ecosystem_workflow.docx — workflow gerado no Claude ▪ Git: commits + push realizados ▪ Storybook 10.3 disponível (upgrade pendente)
--	--

1.2 Problema Conhecido: pnpm run storybook

Bug: pnpm passa args extras ao script, causando conflito. Workaround atual:

```
# NAO funciona:pnpm run storybook --port 6006 --host 0.0.0.0# FUNCIONA  
(workaround atual):sh node_modules/.bin/storybook dev -p 6006 --host  
0.0.0.0# FIX permanente (adicionar ao package.json raiz):"storybook":  
"storybook dev -p 6006 --host 0.0.0.0"# (sem -- no uso, args hardcoded no  
script)
```

2. COMO SALVAR OS CHATLOGS DO CLAUDE

Problema: O Claude.ai não exporta chatlogs automaticamente. Mas eles são seu maior ativo de contexto para continuar o trabalho com OpenCode.

2.1 Metodo A – Copiar manualmente (agora)

1. No Claude.ai: selecionar tudo na conversa (Ctrl+A ou scroll manual)
2. Colar num arquivo .md no repo
3. Nomear como: **docs/chatlogs/claude-YYYY-MM-DD-[tema].md**
4. git add + commit + push

Estrutura recomendada para o repo:

```
storybook-genui/ docs/ chatlogs/
claude-2026-05-09-ecosystem-workflow.md # <-- esta sessao
claude-2026-05-09-storybook-setup.md # sessao OpenCode decisions/
ADR-001-rollup-wasm-patch.md # Architecture Decision Record
ADR-002-mobile-first-default.md research/ ux-friction-log.md
# dores registradas durante dev
```

2.2 Metodo B – Export via API (automatico)

Filosofema: O chatlog que não é salvo é conhecimento que volta ao caos. Todo insight é um ADR em potencial.

```
# Script bash para salvar chatlog do Claude via browser devtools:# 1. Abrir
devtools no Quetta (menu > Mais ferramentas > DevTools)# 2.
Console:copy(document.querySelector('[data-testid="conversation"]')?.innerTe
xt)# 3. Colar no arquivo e salvar# Alternativa: extensao browser
"MarkDownload" ou "SingleFile"# salva a pagina inteira como .md ou .html# No
Termux, apos salvar:cd /home/sukata/storybook-genuigit add docs/chatlogs/git
commit -m "docs: chatlog claude $(date +%Y-%m-%d)"git push
```

2.3 Metodo C – GitHub Issues como chatlog vivo

- Criar um **GitHub Issue** por sessão de trabalho
- Título: **[SESSAO] 2026-05-09 — Storybook Setup + Card Primitive**
- Body: resumo das decisões, links para commits, próximos passos
- Labels: **session, dx, ux-research, primitives**
- Isso cria um **grafo de conhecimento navegável no GitHub**

Metacritica: GitHub Issues como ADR e chatlog tem vantagem: são linkados aos commits via **#issue-number**. Desvantagem: não tem markdown rico como o Claude gera. Melhor prática: Issue como índice + arquivo .md no repo como conteúdo.

3. WORKFLOW GRADUAL — COMO FAZER AOS POUCOS

Formalismo: $W = \{S_1, S_2, \dots, S_n\}$ onde cada S_i é uma sessão de trabalho com pre-condicoes P_i , entregaveis E_i e criterio de aceitacao A_i . A invariante é: $S_i.A = S_{i+1}.P$ (saida de uma é entrada da proxima).

3.1 Principio: Uma Sessao = Um Entregavel Verificavel

Anti-pattern: sessao longa sem entregavel claro → contexto perdido → retrabalho

Pattern correto: sessao curta (30-90min) com **um componente + story + commit + issue fechado**

DX = UX aplicado: assim como o usuario precisa de feedback imediato a cada acao, o dev precisa de **feedback loop curto** — build verde, story no browser, commit pushado.

3.2 Sessoes Recomendadas (proximas)

S₁ — Fix Script + Upgrade (30min) 5. Fix pnpm run storybook (hardcode args) 6. Upgrade Storybook 8→10 (npx storybook upgrade) 7. Confirmar 3 stories no browser 8. Commit: fix(storybook): script + upgrade A₁: localhost:6006 OK com pnpm run storybook S₂ — 6 Primitives (60min cada) 9. MetricGauge (SVG circular, reutilizavel) 10. RitualTimer (Pomodoro, touch-friendly) 11. AgentStateBadge (status, variantes) 12. TagCloud (chips clicaveis, overflow scroll) 13. TimelineBar (horizontal, touch scroll) 14. ScoreBoard (ranking, animado) A₂: cada primitive passa em 360, 414, 768px	S₃ — GraphQL Schema (45min) 15. Schema SDL: Component, Intent, Layout 16. GraphQL Codegen configurado 17. Types exportados de @genui/graphql A₃: tsc compila sem erros S₄ — GenUI Stub (60min) 18. Orchestrator stub com Anthropic API 19. Prompt constitucional implementado 20. Story: GenUI/Playground (input → schema) A₄: story mostra schema gerado por prompt S₅ — PWA + Offline (60min) 21. Service Worker com Workbox 22. Dexie setup para cache de schemas 23. Manifest + icons A₅: app instala no Android como PWA
---	---

4. STEERING PROMPTS PARA OPENCODE

ELI5: Steering prompts são o briefing que você dá ao OpenCode no início de cada sessão — como um standup que orienta o agente sem microgerenciar.

Filosofema: Um steering prompt é uma função de projeção $P: R_w \rightarrow R'$ que colapsa o espaço de respostas possíveis ao subespaço de respostas úteis. Não diz O QUE fazer — diz EM QUE ESPAÇO operar.

4.1 SYSTEM PROMPT BASE (colar no OpenCode)

```
Voce e o arquiteto do STORYBOOK-GENUI - monorepo mobile-first,offline-first, AI-first em compilatorum/storybook-genui.INVARIANTES (nunca viole):- Mobile-first: 360px default, touch targets 44px+, sem overflow- TypeScript strict: sem 'any', sem tipos faltando- Storybook: todo componente tem .stories.tsx com 3 viewports- pnpm workspaces: imports via @storybook-genui/*- Turborepo: tasks em turbo.json- Offline-first: mutacoes vao para IndexedDB antes da rede- UX=DX: se algo e irritante de implementar, documente como friction logAO INICIAR SESSAO:1. Leia o ultimo commit: git log --oneline -52. Leia o TODO.json (ou GitHub Issues abertos)3. Escolha UM entregavel para a sessao4. Ao terminar: commit + push + feche o IssueFORMATO DE SAIDA:- Comece: "SESSAO [N] | [ENTREGAVEL] | [STATUS]"- Termine: "COMMIT: [mensagem] | ISSUES: #N fechados | PROXIMO: [S_{N+1}]"
```

4.2 Steering por Tipo de Tarefa

Criar Primitive (replicar para cada componente)

```
STEERING: Primitive | [NOME] | @storybook-genui/uiOBJETIVO: Componente base reutilizavel por todos os projetos do monorepo.REGRAS:- Props: < 6, todas tipadas, sem 'any'- Zero logica de negocio (apenas apresentacao)- Variantes via prop 'variant', nao via CSS condicional- Story deve ter: Default, Variants, MobileS (360px), Ally- Touch target minimo: 44x44px- Exportar de packages/ui/src/index.tsENTREGUE: [NOME].tsx + [NOME].stories.tsx + types em index.tsFRICTION LOG: anote qualquer ponto onde a API de props ficou confusa
```

UX Research inline (usar durante qualquer sessao)

```
STEERING: UX-Research | Sessao [N]Durante a implementacao, documente em docs/research/ux-friction-log.md:- FRICTION: qualquer momento de confusao ou lentidao- INSIGHT: padroes de uso emergentes- QUESTION: perguntas para testar com usuario real- DECISION: escolha de API ou layout com rationaleFormato: "## [NOME_COMPONENTE] - [DATA]### FRICTION### INSIGHT### QUESTION### DECISION"Isso vira base para UX research qualitativo.
```

Refactor / Eliminacao de Redundancia

```
STEERING: Refactor | Redundancia detectadaComponentes: [A] e [B]Motivo de
suspeita: [semantica similar / API quase identica]Analise: 1. Diferencas
semanticas genuinas? (se sim, documente e encerre) 2. Merge em
shared/[NOME_NOVO] e redirecione imports? 3. Deprecar [B] com jsdoc
@deprecated + migration guide?Entregue: analise escrita + PR com diff
minimoCriterion: tsc + storybook build sem erros apos refactor
```

Bridge Claude → OpenCode (sessao especial)

```
STEERING: Bridge | Claude-Chatlog → OpenCodeArquivo:
docs/chatlogs/claude-[DATA]-[TEMA].mdTarefa: 1. Extrair decisoes
arquiteturais do chatlog 2. Criar ADR (Architecture Decision Record) em
docs/decisions/ 3. Criar/atualizar GitHub Issues para pendencias 4.
Atualizar TODO.json com novos itens 5. Commit: "docs: ADR + issues from
claude chatlog [DATA]"Isso garante que o conhecimento do Claude nao se
perde.
```

5. METACRITICA — ALTERNATIVAS MELHORES E MODERNAS

Filosofema: A metacritica nao e destruicao — e o processo de descascar o acidente para revelar a substancia. Toda escolha tecnica tem um custo oculto; nomeá-lo e o primeiro passo para geri-lo.

5.1 Storybook vs Alternativas

Storybook (escolha atual) ▪ Pro: ecossistema maduro, addons viewport + a11y + interactions ▪ Pro: autodocs = spec viva ▪ Pro: Module Federation via refs ▪ Con: pesado para monorepo grande — startup lento no Termux ▪ Con: configuracao complexa com pnpm workspaces ▪ Con: versoes de addon podem dessincronizar (ja aconteceu)	Alternativas modernas ▪ Ladle: Storybook-like, 10x mais rapido, Vite nativo, ideal para Termux ▪ Histoire: Vue-first mas suporta React, UI moderna ▪ Vitebook: experimental, mais leve ▪ Recomendacao: manter Storybook agora, avaliar Ladle em S₂ se build > 30s no Android
---	---

5.2 Monorepo: pnpm+Turbo vs Alternativas

pnpm + Turborepo (atual) ▪ Pro: disk-efficient, strict peer deps ▪ Pro: Turborepo: cache de tasks, paralelismo ▪ Con: Rollup WASM patch (fragil, pode quebrar em upgrade) ▪ Con: pnpm arg mangling (bug conhecido)	Alternativas ▪ Nx: mais completo, generators, affected-graph, melhor CI ▪ Bun workspaces: 3x mais rapido que pnpm, mas ARM64 experimental ▪ moon: Rust-based, muito rapido, novo mas promissor ▪ Recomendacao: pnpm+Turbo ok por ora; migrar para Nx em escala (>10 pacotes)
---	--

5.3 Estado: Zustand vs Alternativas

Zustand (escolha atual) ▪ Pro: simples, sem providers, DevTools ▪ Pro: atomic por slice ▪ Con: sem normalização de dados de servidor	Alternativas ▪ Jotai: atomic primitivo, menor bundle, React Suspense nativo ▪ Nanostores: framework-agnostico, 334 bytes, ideal para SGUI multi-framework ▪ TanStack Query: para server state, combinar com Zustand para UI state ▪ Recomendacao: Zustand + TanStack Query — separar server state de UI state
--	--

5.4 UX Research — Práticas Modernas

Problema atual: UX research esta sendo feito implicitamente (friction log). Tornar explicito multiplica o valor.

Metodos leves (para solo dev)

- **Friction log:** anotar dores durante dev (DX=UX)
- **5-second test:** mostrar story por 5s para alguem — o que eles viram?
- **Think-aloud:** gravar tela enquanto usa o Storybook e verbaliza
- **Cognitive walkthrough:** simular usuario em cada story

Artefatos de UX no repo

- **docs/research/personas.md:** quem usa cada componente
- **docs/research/user-flows.md:** jornadas por projeto
- **docs/research/ux-friction-log.md:** dores durante dev
- **Storybook autodocs:** documentacao como artefato de UX
- **A11y addon CI check:** acessibilidade como gate de qualidade

6. INSTRUÇÕES PARA VOCE — PASSO A PASSO

Esta seção é para você, não para o OpenCode. Siga a ordem.

6.1 Agora (antes de fechar o Claude)

24. **Salvar este documento:** Download do .docx gerado
25. **Salvar o chatlog desta sessão:**
 - Selecionar toda a conversa no Claude.ai
 - Salvar como: [docs/chatlogs/claude-2026-05-09-storybook-workflow.md](#)
 - git add + commit + push
26. **Salvar os documentos anteriores:**
 - [ecosystem_workflow.docx](#) → já disponível
 - session-genUI.md (já tem)
 - PLANO_ENGENHARIA_PROMPTS.md (já no repo)
27. **Criar GitHub Issues** para as sessões S₁–S₅ do roadmap

6.2 Próxima Sessão — Checklist de Início

```
# 1. Entrar no ambiente proot-distro login ubuntu# 2. Ir ao projeto cd /home/sukata/storybook-genui# 3. Ver estado atual git log --oneline -5 cat TODO.json # ou gh issue list# 4. Subir Storybooksh node_modules/.bin/storybook dev -p 6006 --host 0.0.0.0 &# Abrir localhost:6006 no Quetta# 5. Abrir OpenCode com steering prompt da sessão opencode# Colar o SYSTEM PROMPT BASE + steering da sessão específica# 6. Ao terminar: git add -A && git commit -m "feat(ui): [COMPONENTE] + story" git push# Fechar Issue correspondente no GitHub
```

6.3 Como Usar Este Documento com o OpenCode

- **Opção A — Anexar direto:** Se OpenCode aceitar anexo, subir este .docx como contexto
- **Opção B — Converter para .md:**

```
# Converter este docx para md (no ubuntu proot): pandoc CONTINUIDADE_WORKFLOW.docx -o docs/CONTINUIDADE_WORKFLOW.md git add docs/CONTINUIDADE_WORKFLOW.md && git commit -m "docs: continuidade workflow"# No OpenCode: "Leia docs/CONTINUIDADE_WORKFLOW.md e siga as instruções"
```

- **Opção C — System prompt:** Copiar a seção 4.1 deste doc e colar como system prompt no OpenCode

6.4 Convenções Git para Este Projeto

```
# Tipos de commit (Conventional Commits): feat(ui): novo componente MetricGauge + storyfix(storybook): script pnpm run storybook arg mangling docs(chatlog): sessão claude 2026-05-09 refactor(ui): merge Badge e StatusBadge em shared test(ui): story ally check MetricGauge chore(deps): upgrade storybook 8 -> 10# Branch strategy: main # sempre verde,
```

```
stories passandofeat/[nome]    # feature branch por sessaofix/[bug]    #
hotfix# Issues GitHub:[S1] Fix pnpm storybook script + upgrade Storybook
10[S2] Primitive: MetricGauge[S2] Primitive: RitualTimer... etc
```

7. ROADMAP CONSOLIDADO — VISAO GERAL

Camada 1 — Fundacao (atual) ▪  Storybook rodando no Android ▪  Monorepo pnpm+Turbo ▪  Card primitive + story ▪  Fix pnpm script ▪  Upgrade Storybook 10 ▪  6 primitives restantes Camada 2 — SGUI (S₃) ▪ ○ GraphQL schema SDL ▪ ○ Resolvers gerando schemas ▪ ○ SGUIRenderer universal ▪ ○ Apollo Client configurado	Camada 3 — GenUI (S₄) ▪ ○ Orchestrator LLM (Anthropic API) ▪ ○ Prompt constitucional ▪ ○ Story GenUI/Playground ▪ ○ Fallback offline Camada 4 — PWA + Emacs (S₅) ▪ ○ Service Worker Workbox ▪ ○ Dexie + Sync Manager ▪ ○ Manifest + install Android ▪ ○ Emacs bridge (experimental)
---	---

◆ CODA — O QUE FICOU PROVADO HOJE

O screenshot nao mente: Storybook rodando no browser Quetta via localhost:6006, viewport Mobile S 360x640, story Card/Mobile Default ativa. Termux → proot-distro Ubuntu → Node → React → Android. O pipeline funciona.

O que fica para amanha nao e duvida tecnica — e trabalho iterativo. Cada sessao e um componente, cada componente e uma story, cada story e uma prova de que UX=DX.

$\Pi(S_1 \rightarrow S_5)$ = Storybook vivo no Android, schemas gerados por intenção, offline-first, com cada decisão rastreada em chatlog, ADR e Issue.